

1. What is Git?

Git is a Distributed Version Control System (DVCS).

It helps developers:

- Track code changes
- Collaborate with teams
- Restore previous versions
- Manage multiple features simultaneously

Example

Suppose today your Angular application works perfectly.

Tomorrow you make changes and the app breaks.

With Git you can:

`git checkout previous-version`

and restore the working code.

2. What is GitHub?

GitHub is a cloud platform that hosts Git repositories.

Think:

Git	GitHub
Tool installed on PC	Website
Tracks code	Stores code online
Version control	Collaboration
Local repository	Remote repository

Example

Git = Notebook on your laptop

GitHub = Google Drive storing that notebook online

Official site:

[GitHub](https://github.com)

3. Git Architecture

Working Directory

↓

git add

↓

Staging Area

↓

git commit

↓

Local Repository

↓

git push

↓

GitHub Repository

4. Git Workflow

Code

↓

git add

↓

git commit

↓

git push

↓

GitHub

5. Important Git Terms

Repository (Repo)

Project folder tracked by Git.

Example:

AngularApp/

Commit

Snapshot of code.

Example:

```
git commit -m "Login page completed"
```

Branch

Separate line of development.

Example:

```
main  
develop  
testing  
feature-login
```

Merge

Combines branches.

Example:

```
git merge develop
```

Clone

Download repository.

```
git clone https://github.com/user/project.git
```

Pull

Get latest changes.

```
git pull
```

Push

Upload code.

```
git push
```

6. Installation

Install Git

Download:

[Git Download](#)

Verify:

```
git --version
```

Output:

```
git version 2.45.0
```

7. Configure Git First Time

```
git config --global user.name "Ibrahim Patel"
```

```
git config --global user.email "ibrahim@gmail.com"
```

Check:

```
git config --list
```

8. Create GitHub Repository

Step 1

Login GitHub

Step 2

Click

New Repository

Step 3

Repository Name:

AngularEmployeeApp

Step 4

Create Repository

GitHub provides:

```
git remote add origin https://github.com/user/repo.git
```

9. Push Project First Time

Move to project folder:

```
cd AngularEmployeeApp
```

Initialize:

```
git init
```

Add files:

```
git add .
```

Commit:

```
git commit -m "Initial Commit"
```

Connect GitHub:

```
git remote add origin https://github.com/user/repo.git
```

Push:

```
git push -u origin main
```

10. How Git Works Internally

Example:

File:

```
employee.component.ts
```

Changed:

```
name="John";
```

Git creates hash:

```
a1b2c3d4
```

Commit:

```
git commit -m "Added employee name"
```

Git stores:

Hash

Author

Date

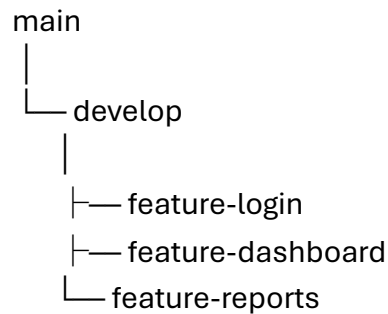
Changes

Every commit gets unique ID.

Example:

```
6d5c4b1
```

11. Branch Strategy (Industry Standard)



12. Production, Testing, Develop Branches

Main Branch

Production

Live application.

Example:

www.company.com

Testing Branch

QA/UAT Testing

Before production.

Develop Branch

Developer Integration Branch

All completed features merge here.

13. Create Branches

Create Develop:

`git checkout -b develop`

Push:

```
git push -u origin develop
```

Create Testing:

```
git checkout -b testing
```

Push:

```
git push -u origin testing
```

14. Feature Branch Example

Create:

```
git checkout develop
```

```
git checkout -b feature-login
```

Work.

Commit.

```
git add .
```

```
git commit -m "Login Feature Completed"
```

Push:

```
git push origin feature-login
```

15. Merge Process

Feature → Develop

```
git checkout develop
```

```
git merge feature-login
```

Develop → Testing

git checkout testing

git merge develop

Testing → Main

git checkout main

git merge testing

16. Angular Project Example

Create Angular Project:

ng new EmployeePortal

Move:

cd EmployeePortal

Initialize Git:

git init

Create repository on GitHub.

Connect:

git remote add origin <https://github.com/user/EmployeePortal.git>

Push:

git add .

git commit -m "Angular Initial Project"

git branch -M main

git push -u origin main

Now Angular project is on GitHub.

17. Visual Studio 2022 Git Integration

Visual Studio 2022 has built-in Git support.

Open Project

Visual Studio



Git Menu



Create Git Repository

Commit

Git Changes



Message



Commit All

Push

Git Changes



Push

Pull

Git



Pull

Create Branch

Git



New Branch

Name:

develop

Create

18. Common Git Commands

Initialize

git init

Status

git status

Shows modified files.

Add All

git add .

Commit

git commit -m "Message"

Push

git push

Pull

git pull

Clone

git clone URL

Branch List

git branch

Create Branch

git checkout -b develop

Switch Branch

git checkout develop

Modern:

git switch develop

Merge

git merge develop

Delete Branch

git branch -d feature-login

Commit History

git log

One Line Log

git log --oneline

Remote Repositories

git remote -v

Fetch

git fetch

Downloads changes without merging.

Stash

git stash

Temporary save work.

Restore:

git stash pop

19. Most Asked Interview Questions

What is Git?

Version control system that tracks code changes.

Difference Between Git and GitHub?

Git = Version control tool

GitHub = Cloud repository hosting platform

What is Commit?

Snapshot of project changes.

What is Branch?

Independent line of development.

What is Merge?

Combining branches.

Difference Between Fetch and Pull?

Fetch:

`git fetch`

Downloads changes.

Pull:

`git pull`

Downloads + merges.

What is Stash?

Temporarily saves unfinished work.

What is Rebase?

Moves commits to another base branch.

`git rebase develop`

Creates cleaner history.

What is Conflict?

When Git cannot decide which change to keep.

Example:

Developer A:

name="John";

Developer B:

name="Mike";

Git asks for manual resolution.

20. Real Company Workflow

main
|
testing
|
develop
|
feature branches

Flow:

Feature Branch

↓

Develop

↓

Testing

↓

Main

Example:

feature-login

↓

develop

↓

testing

↓

main

21. Complete Daily Developer Workflow

Morning:

`git pull origin develop`

Create branch:

`git checkout -b feature-profile`

Coding...

`git add .`

`git commit -m "Profile API Integration"`

Push:

`git push origin feature-profile`

Create Pull Request in GitHub.

After approval:

`feature-profile → develop`

22. Git Cheat Sheet (Must Remember)

`git init`

`git clone URL`

`git status`

`git add .`

`git commit -m "msg"`

`git push`

`git pull`

`git fetch`

`git branch`

`git checkout branch`

`git checkout -b branch`

`git switch branch`

`git merge branch`

`git rebase branch`

git log
git log --oneline
git stash
git stash pop
git remote -v
git branch -d branch

1. Pull Request (PR)

This is one of the most important GitHub concepts.

What is Pull Request?

A Pull Request (PR) is a request to merge code from one branch into another.

Example:

feature-login
↓
Pull Request
↓
develop

Interview Answer

A Pull Request is used for code review before merging changes into another branch. Team members review the code, suggest changes, and approve it before merging.

2. .gitignore

Very important in Angular projects.

Example:

node_modules/
dist/
.vscode/
.env

Without .gitignore, thousands of unnecessary files get pushed.

Angular Example

node_modules/
dist/
coverage/

3. Git Reset

Used to undo commits.

Soft Reset

`git reset --soft HEAD~1`

Removes commit but keeps changes.

Hard Reset

`git reset --hard HEAD~1`

Removes commit and changes.

Interviewers often ask this.

4. Git Revert

Safer than reset.

`git revert commit-id`

Creates a new commit that undoes previous changes.

Used in production systems.

5. Difference Between Merge and Rebase

Merge

`git merge develop`

Creates merge commit.

History:

A---B---C

\

D---E

\

M

Rebase

git rebase develop

History:

A---B---C---D---E

Cleaner history.

Interview Question

When do you use Rebase?

Answer:

Rebase is used to maintain a clean linear commit history before merging branches.

6. Tags (Release Versions)

Example:

git tag v1.0

git push origin v1.0

Used for releases.

Example:

v1.0

v1.1

v2.0

7. Git Conflict Resolution

Interviewers ask this frequently.

Example:

Developer A:

```
name="John";
```

Developer B:

```
name="Mike";
```

Git shows:

```
<<<<<<< HEAD
```

```
John
```

```
=====
```

```
Mike
```

```
>>>>>> feature
```

Resolve manually:

```
name="John Mike";
```

Then:

```
git add .
```

```
git commit
```

8. Git Flow Diagram

A more realistic workflow:

```
main
```

```
|
```

```
|— release
```

```
|
```

```
|— testing
```

```
|
```

```
|— develop
```

```
|  
|— feature-login  
|— feature-dashboard  
|— feature-report
```

9. GitHub Actions (CI/CD)

Modern interviews often ask:

What is GitHub Actions?

Automation tool inside GitHub.

Example:

Push Code

↓

Build Angular App

↓

Run Tests

↓

Deploy

10. Cherry Pick

Useful in production support.

`git cherry-pick commit-id`

Copies one specific commit.

Example:

develop → 10 commits

Need only 1 bug fix

Cherry-pick that commit.

For Angular Developers Specifically

Always push these files:

```
src/  
angular.json  
package.json  
package-lock.json  
tsconfig.json
```

Never push:

```
node_modules/  
dist/  
coverage/  
  
.gitignore  
  
node_modules/  
dist/  
coverage/
```

Interview Answer (2-Minute Version)

Git is a distributed version control system used to track source code changes and collaborate with teams. GitHub is a cloud-based platform used to host Git repositories. In my projects, I usually follow a branching strategy with main, testing, and develop branches. I create feature branches from develop, commit changes locally, push them to GitHub, and create pull requests. After code review, changes are merged into develop, tested in testing, and finally deployed through main. I regularly use commands such as git clone, git pull, git add, git commit, git push, git merge, git rebase, and git stash.

This explanation is strong enough for most Angular, .NET, Java, Full Stack, and DevOps interviews.

If an Interviewer Asks: "Explain Git Workflow"

A strong answer is:

In my projects, I use Git for version control and GitHub as the remote repository. We follow a branching strategy with main, testing, and develop branches. New features are created from develop using feature branches. After development, changes are committed locally, pushed to GitHub, and a Pull Request is created. After code review and approval, the

feature branch is merged into develop. The testing team validates changes in the testing branch, and finally stable code is merged into main for production deployment.